Fix our mistake here and use a matcher whose failure message clearly spells out the difference between the two collections.

Working for the Weekend

For our final example, we're going to write a calendar-related spec that determines whether any given day is on the weekend:

11-matchers-included-in-rspec-expectations/exercises/calendar_spec.rb

```
RSpec.describe Calendar do
  let(:sunday_date) { Calendar.new('Sun, 11 Jun 2017') }

  it 'considers sundays to be on the weekend' do
    expect(sunday_date.on_weekend?).to be true
  end
end
```

Here's an obviously incorrect implementation of the `on_weekend?` method:

11-matchers-included-in-rspec-expectations/exercises/calendar_spec.rb

```
require 'date'

Calendar = Struct.new(:date_string) do
  def on_weekend?
    Date.parse(date_string).saturday?
  end
end
```

When you run this spec, you get the stilted phrase "`to be true`" in the output. Change this matcher to one that reads more clearly in the test report.

Bonus Points

As we mentioned earlier, the main point of these exercises was to practice using matchers that express just what you want to say about your code's behavior (and no more), and that give you clear output.

So, there's no need to fix the underlying implementations. But for extra credit, feel free to make the specs pass. Post your solutions in the forums, and we'll send you a GIF of a gold star.

Either way, meet us in the next chapter to see how you can create your own matchers that are just as expressive as the built-in ones.